AEP Agentic-Container OS — Low-Level Design

LLD · v1.1 · Status: Proposed · Branch: `docs/dual-lane-governance-proposal` · Companion: `aep-hld-v1.1.html`
Module path: `github.com/pratik-saptarshi/autonomous-engineering-platform` · Go 1.27.x · stdlib + `golang.org/x/sys` only unless stated

1. Scope and conventions

This document specifies interfaces, data formats, algorithms and an ordered task list for turning the as-built `aep` binary into the dispatching, admission-controlled substrate described in the HLD. Every item is marked AS-BUILT (exists in `develop`) or NEW (to be written). Nothing marked *new* exists yet.

- Process boundary rule: lanes are containers; the `aep` binary on the substrate is the only component that starts, tracks and stops them.
- All persistent state is JSON written atomically (temp file + rename) under `$AEP_STATE_DIR` (default `~/.local/state/aep`; on Windows `%LOCALAPPDATA%\aep`), mode 0700.
- Exit codes: `0` ok · `1` failed check · `2` usage · `3` admission refused · `4` degraded/quarantined · `75` reserved for `ghg` budget denial (passed through unchanged).

2. Repository layout

```
cmd/aep/main.go                        as-built  subcommand switch; flags per subcommand
internal/platform/memory/              as-built  GetMemoryInfo() — linux /proc/meminfo,
internal/platform/lock/                as-built  FileLock — unix.Flock / LockFileEx; Try
internal/platform/process/             as-built  ProcessGroup — Setpgid + group signal /
internal/verifier/runner/              as-built  DiscoverCommands, RunVerification, Tail
internal/profile/                      new       hardware profile loading + validation
internal/policy/                       new       policy.data parse + committed-blob hash
internal/ledger/                       new       beads adapter (issue read/claim/close)
internal/memoryd/                      new       NeuroStrata client (search, inscribe, s
internal/admission/                    new       memory/CPU reservation accounting
internal/lane/                         new       lane manifest, claim/release/reap, cont
internal/dispatch/                     new       tier routing + degradation decisions
internal/factory/                      new       thin ESF client (wraps the `factory` CL
cmd/aep  (new subcommands)             new       dispatch · lane · profile · boot --fact
deploy/colima/                         new       colima.yaml + substrate compose file
docs/plan/                             this doc  HLD + LLD (html + pdf)
```

3. As-built components (reference)

Package / command	Behaviour that later work depends on
`memory.GetMemoryInfo()`	Returns `MemoryInfo{TotalBytes, AvailableBytes}`. Darwin availability is free + speculative pages only, so it under-reports reclaimable memory; admission must not treat it as exact.
`lock.New(path)`	Advisory exclusive lock. `TryLock()` returns `ErrLocked` if held. The kernel releases it on process death — the property the lane fence relies on.
`process.NewCommand(ctx,name,args,cfg)`	Returns a `ProcessGroup`; `Kill()` terminates the whole tree (group SIGTERM→grace→SIGKILL on POSIX, Job Object on Windows).
`runner.RunVerification(ctx,root,minMem,perCmd,budget)`	Discovers go/cargo/npm/pytest/make checks, runs them sequentially bounded by timeout and budget, stops at first failure, returns `VerificationResult{Pass, Receipts[]}` with 64 KiB tail hashes.
`aep boot`	Memory probe, 384 MiB floor, platform line, then `BOOT-GATE-RESULT: SYSTEM-OK`. **Does not** yet check NeuroStrata, guards, gh auth, binaries or policy.
`aep supervisor [--loop --interval D --once]`	Tick = boot gate then one `RunVerification`. Boot-gate failure is fatal in every mode; verification failure is fatal only outside loop mode. Ticks never overlap.

4. New packages — interfaces

4.1 `internal/profile`

```
type Profile struct {
    Name            string // "nano-8g", "small-16g", "medium-32g", "workstation-48g"
    VMCPUs          int    // 2 (nano) up to 12 (workstation)
    VMMemoryMiB     uint64 // 8192 up to 49152
    VMDiskGiB       int    // 30 up to 220
    SubstrateMiB    uint64 // 2560 up to 8192
    Tier1BudgetMiB  uint64 // 4096 up to 16384
    Tier2BudgetMiB  uint64 // 0 (nano) up to 24576 (workstation)
    Tier1LaneMiB    uint64 // default per-lane reservation, 2048
    Tier2LaneMiB    uint64 // default per-pipeline reservation, 4096 - 6144
    MaxTier1Lanes   int    // hard ceiling: 2 (nano) up to 10 (workstation)
    MaxTier2Lanes   int    // hard ceiling: 0 (nano) up to 6 (workstation)
    Tier2Supported  bool   // true only if /dev/kvm or nested virt is detected
    MemFloorMiB     uint64 // 384 (nano) / 512
    ONNXThreads     int
}
func Detect() (Profile, error)            // probes CPU, RAM, disk, /dev/kvm, and conta
func Load(path string) (Profile, error)   // KEY=VALUE file, same keys as hardware-prof
func (p Profile) Validate() error         // sum(substrate+t1+t2) <= VMMemoryMiB; ceili
```

Presets include `nano-8g` (Intel N100 / Cloud 2vCPU / RPi 5; Tier 2 disabled), `small-16g` (i5/Ryzen 5; 1 pipeline), `medium-32g` (i7/Ryzen 7; 2–3 pipelines), and `workstation-48g` (M3 Max Colima / Threadripper; 4–6 pipelines).

4.2 `internal/policy`

```
type Policy struct { EnvScrubFragments []string; SensitivePaths []string; Raw []byte; S
func Load(path string) (Policy, error)
func (p Policy) VerifyAgainstHEAD(repoDir string) error   // git show HEAD:scripts/poli
func (p Policy) ScrubEnv(env []string) []string          // drops names containing any
func (p Policy) CheckWorkingSet(paths []string) error    // refuses sensitive-path matc
```

Behavioural parity target: `scripts/boot-gate.sh` `check_policy` and `lane-coord.sh`'s working-set check. When `policy.data` is absent, return a distinguishable `ErrPolicyInactive` so boot can print `POLICY-INACTIVE` (exit 0) instead of pretending a check passed.

4.3 `internal/ledger` (beads adapter)

```
type Issue struct { ID string; Title string; Status string; Tier string /* "local"|"fac
type Ledger interface {
    Next(ctx context.Context) (*Issue, error)          // highest-priority open issue w
    Get(ctx context.Context, id string) (*Issue, error)
    Claim(ctx context.Context, id, owner string) error  // sets in_progress; idempotent
    Close(ctx context.Context, id, reason string) error
}
func NewBD(binary string) Ledger   // shells out to `bd ... --json`; no direct Dolt acc
```

The adapter only parses `bd`'s JSON output. Tier comes from a label (`tier:local` / `tier:factory`); a missing label is treated as `local` only if the issue also carries `risk:low`, otherwise dispatch refuses

(exit 4) so untrusted work cannot silently land in the weaker lane.

4.4 `internal/memoryd` (NeuroStrata host relay)

```
type Client interface {
    Status(ctx context.Context) error                                    // maps to `neu
    Search(ctx context.Context, ns, query string, k int) ([]Hit, error)
    Inscribe(ctx context.Context, ns string, rec Record) error
    Guards(ctx context.Context) (n int, hasDestructiveRule bool, err error)
}
type RelayServer interface {
    ServeLaneSocket(ctx context.Context, laneID string, sockPath string) error
}
func NewCLI(binary string) Client   // wraps the neurostrata-mcp CLI, as boot-gate.sh d
```

Host supervisor exposes a per-lane UNIX domain socket (`/run/aep/lanes/<laneID>/memory.sock`) mounted into the lane container. The relay intercepts requests, strips credentials, attaches the lane provenance tag, and proxies to NeuroStrata on `127.0.0.1:34343` . MicroVMs use an outbox spool file ingested upon run completion, preserving zero-egress boundaries.

4.5 `internal/gateway` (ghg IPC relay)

```
type GHGRelay interface {
    ServeLaneSocket(ctx context.Context, laneID string, sockPath string) error
}
type GHGClient interface {
    Watch(ctx context.Context, runID string) error
    API(ctx context.Context, method, endpoint string, body io.Reader) ([]byte, error)
    PRView(ctx context.Context, prNumber int) ([]byte, error)
}
```

Overcomes the fact that `tools/ghg/ghg.py` is a CLI with no network daemon. The host-side `aep gwd` daemon serves an IPC socket (`/run/aep/lanes/<laneID>/ghg.sock`). Inside the lane, a lightweight shim forwards commands to the socket. The host relay executes `ghg.py` locally, enforcing the 3,500 req/h cap and poller coalescing. GitHub PAT credentials never enter the container.

4.6 `internal/admission`

```
type Request struct { LaneID string; Tier int; MemMiB uint64; CPUs float64 }
type Reservation struct { LaneID string; Tier int; MemMiB uint64; CPUs float64; At time
type Controller struct { /* profile, state file, lock */ }
func (c *Controller) Admit(ctx context.Context, r Request) (Reservation, error)  // Err
func (c *Controller) Release(laneID string) error
func (c *Controller) Snapshot() (Usage, error)
```

State lives in `$AEP_STATE_DIR/admission.json` , guarded by one `lock.FileLock` held only for the read-modify-write. See §6 for the algorithm.

4.7 `internal/lane`

```
type Manifest struct {
    LaneID, IssueID string
    Tier            int
    Image, Container string
    Worktree        string
    ReservedMiB     uint64
    LauncherPID     int
    StartedAt       time.Time
    State           string // claimed|running|verifying|done|failed|reaped
}
type Runtime interface {                      // docker/podman-compatible CLI driver
    Run(ctx context.Context, spec RunSpec) (containerID string, err error)
    Stop(ctx context.Context, id string, grace time.Duration) error
    Inspect(ctx context.Context, id string) (State, error)
}
func Claim(ctx context.Context, d Deps, issue Issue) (Manifest, error)
func Release(ctx context.Context, d Deps, laneID string, clean bool) error
func Reap(ctx context.Context, d Deps) ([]string, error)   // lanes whose launcher is d
```

The fence is a `lock.FileLock` on `lanes/<id>.fence` held by the launching `aep lane claim --exec` process for the lane's lifetime, mirroring `lane-coord.sh`. A takeover or `release --clean` requires both: fence free *and* recorded container no longer running.

4.8 `internal/dispatch`

```
type Decision struct {
    IssueID string
    Tier    int
    Action  string // "run-local" | "run-factory" | "quarantine" | "fallback-local" | "
    Reason  string
}
func Decide(ctx context.Context, d Deps, issue Issue) Decision
```

Decision table in §7. `Decide` is pure given its probes, so it is unit-testable with fakes.

4.9 `internal/factory`

```
type Client interface {
    Doctor(ctx context.Context) error
    Run(ctx context.Context, req RunRequest) (RunHandle, error)   // WorkflowID = "esf-
    Status(ctx context.Context, id string) (RunStatus, error)
    PatchPath(ctx context.Context, id string) (string, error)
    VerifyEgressProbe(ctx context.Context, id string) (bool, error)
}
```

Wraps the `factory` CLI. A guard reads `factory.toml` and rejects `hardening.acknowledge_open_egress = true` (the upstream default is `false`) and ensures closed egress policies. `VerifyEgressProbe` checks the cryptographic evidence digest from the in-VM probe before accepting run evidence.

5. CLI surface (new)

Command	Behaviour	Exits
`aep boot [--factory] [--profile F]`	Extends the as-built gate: binaries (`neurostrata-mcp bd ocr gh`), NeuroStrata status, guard count ≥3 and destructive-actions rule, `gh auth status` , policy hash, memory floor from profile. `--factory` adds `factory doctor` and the egress assertion. Prints the same `BOOT-GATE: <name> ok|FAIL` lines and `BOOT-GATE-RESULT` tokens as the shell gate.	0 / 1
`aep profile show|validate`	Prints the resolved profile and current admission usage.	0 / 1
`aep dispatch [--issue ID] [--dry-run]`	Boot gate → pick issue (or `Next`) → `Decide` → admit → start lane/pipeline. `--dry-run` prints the decision record only.	0 / 3 / 4
`aep lane claim|release|status|reap`	Go port of `lane-coord.sh` 's verbs for container lanes (§4.6).	0 / 1 / 3
`aep supervisor` (extended)	Tick becomes: boot gate → `reap` → dispatch up to the admission ceiling → verification of done lanes. Existing flags unchanged; new `--dispatch` opt-in so current behaviour is preserved by default.	unchanged

6. Admission algorithm

```
Admit(req):
  lock(admission.lock)                          # FileLock.Lock, bounded wait 30s
  s := load(admission.json)
  if req.Tier == Tier2 && !profile.Tier2Supported:
    return ErrTier2Unsupported("hardware virtualization (/dev/kvm) unavailable on this

  budget  := profile.TierBudgetMiB(req.Tier)
  used    := sum(s.Reservations[tier == req.Tier].MemMiB)
  ceiling := profile.MaxLanes(req.Tier)
  host    := memory.GetMemoryInfo().AvailableBytes (inside the VM)

  if count(tier) >= ceiling                          -> ErrNoCapacity("lane ceiling")
  if used + req.MemMiB > budget                      -> ErrNoCapacity("tier memory budg
  if host - req.MemMiB*MiB < profile.MemFloorMiB*MiB -> ErrNoCapacity("host floor")
  append reservation; write atomically; unlock; return
```

- Reservations are keyed by lane ID and removed in `Release` and by `Reap` ; a reservation whose lane manifest is `reaped` is garbage-collected on the next `Admit` .
- The count ceiling and the memory budget are *both* enforced, parameterizing dynamically by the active hardware profile (`nano-8g` to `workstation-48g`).
- Container memory is enforced by the runtime: `--memory=<ReservedMiB>m --memory-swap=<same> --cpus=<n>` , so a lane exceeding its reservation is OOM-killed inside its own cgroup rather than starving the substrate.

- CPU is a soft share (not a hard partition): reservations sum may exceed host cores by design; only memory and KVM availability are admission-blocking resources.

7. Dispatch decision table

Tier tag	Probes	Decision	Exit
`local`	boot ok, admission ok	`run-local`	0
`local`	admission ErrNoCapacity	`wait` (bounded retry, then leave issue open)	3
`factory`	`factory doctor` ok, egress guard closed, probe ok, admission ok	`run-factory`	0
`factory`	Temporal or Cube unreachable	`quarantine`: label issue `blocked:factory-unavailable`, write decision record, do not start anything	4
`factory`	KVM missing / `!Tier2Supported` (e.g. Nano or Cloud VM)	`quarantine`: label issue `blocked:unsupported-tier2-no-kvm`, do not start anything	4
`factory`	`acknowledge_open_egress = true` OR probe failed	`quarantine` (governance failure, never auto-fallback)	4
`factory`	unreachable/unsupported *and* operator provides `--override-tier1-risk-reassessment` *and* `fallback:local-ok`	`fallback-local`: run in a fenced Tier-1 lane with `review:escalated` (mandatory human review)	4
missing, no `risk:low`	—	refuse (cannot place untrusted work)	4

Automatic downgrade is strictly prohibited. An issue tagged `factory` represents untrusted code, external dependencies, or high-blast-radius execution; executing it on the shared Tier-1 host kernel without hardware isolation would defeat the routing rationale. Fallback to Tier 1 requires an explicit, authenticated operator risk-reassessment flag (`--override-tier1-risk-reassessment --reason="..."`) at dispatch time.

8. Data formats

8.1 Lane manifest — `$AEP_STATE_DIR/lanes/<laneId>.json`

```
{
  "schema": 1,
  "lane_id": "t1-7f3a9c",
  "issue_id": "aep-142",
  "tier": 1,
  "image": "aep:1.1",
  "container": "aep-lane-t1-7f3a9c",
  "worktree": "/Volumes/Build 1/Git-SCM/worktrees/aep-142",
  "reserved_mib": 2048,
  "launcher_pid": 41233,
  "started_at": "2026-11-08T23:59:00Z",
  "state": "running",
  "policy_sha256": "…",
  "receipts": "/…/aep-142.receipts.jsonl"
}
```

8.2 Dispatch decision record — `$AEP_STATE_DIR/decisions/<ts>-<issue>.json`

```
{ "schema": 1, "issue_id": "aep-142", "tier_tag": "factory",
  "action": "quarantine", "reason": "temporal unreachable",
  "probes": { "boot": "ok", "factory_doctor": "fail: connection refused", "egress": "ok
  "at": "2026-11-08T23:59:00Z" }
```

8.3 Identity conventions

- Lane ID: `t<tier>-<6 hex>`. Container name: `aep-lane-<laneId>`. Worktree: `$AEP_WORKTREES_ROOT/<issueId>`.
- ESF workflow ID: `esf-beads-<issueId>`; a second submit for the same issue is rejected by the dispatcher before calling `factory run`.
- Branch: `type/scope-short-description` and Conventional Commits, per repo `AGENTS.md`; commit identity per `AGENTS.md`.

9. Runtime topology

9.1 Container and VM runtimes

```
# macOS (M3 Max / Apple Silicon Workstation):
colima start --profile aep \
  --cpu 12 --memory 48 --disk 220 \
  --vm-type vz --mount-type virtiofs

# Linux hosts (Intel N100 / AMD Ryzen / Cloud VMs):
# Runs natively on host cgroups v2 using Podman (rootless) or Docker:
aep boot --profile nano-8g --runtime podman
```

Hardware virtualization (`/dev/kvm`) is probed at boot. If available, Tier-2 microVM sandboxing is enabled; if unavailable (typical of low-power edge nodes and standard cloud VMs), Tier 2 is disabled and all untrusted tasks are safely quarantined.

9.2 Substrate compose file (`deploy/compose/substrate.compose.yml`)

```
services:
  neurostrata:   # image bakes neurostrata-mcp + pre-seeded model cache; HF_HUB_OFFLINE
    volumes: [ns-data:/data]
    mem_limit: 2g
  aep-supervisor:
    command: ["supervisor","--loop","--dispatch","--interval","2h"]
    depends_on: [neurostrata]
    volumes: [aep-state:/state, aep-sockets:/run/aep, "${AEP_REPO}:/repo:ro"]
    mem_limit: 512m
volumes: { ns-data: {}, aep-state: {}, aep-sockets: {} }
```

The substrate supervisor manages two IPC relays in `aep-sockets` : `memory.sock` (proxying requests to NeuroStrata) and `ghg.sock` (proxying GitHub API calls to `ghg.py`).

9.3 Lane container run spec

```
docker run -d --name aep-lane-t1-7f3a9c \
  --memory=2048m --memory-swap=2048m --cpus=1 \
  --read-only --tmpfs /tmp:rw,size=256m \
  --network none \                          # zero direct network egress
  -v "/run/aep/lanes/t1-7f3a9c:/run/aep:rw" \ # mounted IPC socket relays for memory an
  -v "<worktree>:/workspace" -w /workspace \
  --env-file <scrubbed.env> \               # produced by policy.ScrubEnv; no tokens
  aep:1.1 lane-run --issue aep-142
```

> **Zero-egress lane architecture.**
> Lanes run with `--network none` (or an internal isolated bridge) and mount per-lane IPC sockets for memory and GitHub operations. GitHub PAT tokens and loopback ports never enter the container namespace. Dependency caches and package modules are pre-baked into the lane image or mounted via read-only volume cache.

10. Error handling and failure matrix

Failure	Detection	Behaviour
NeuroStrata down at dispatch	`memoryd.Status`	Boot fails closed (exit 1). Supervisor in loop mode treats it as fatal and lets the service manager own retry cadence.
Tier-2 task on host without KVM	`!profile.Tier2Supported`	`quarantine`: labeled `blocked:unsupported-tier2-no-kvm`. Never auto-downgraded to Tier 1.
Egress probe fails or unverified	`VerifyEgressProbe`	`quarantine`: run evidence rejected; PR creation blocked; never merged.
Relay socket communication error	`ECONNREFUSED / EPIPE`	Lane execution pauses; alerts supervisor; never falls back to direct network calls.
Launcher crashes mid-lane	fence free	Lane marked reapable only if the container is also gone; otherwise left running and reported.
Lane exceeds memory reservation	runtime OOM kill (exit 137)	Receipt records failure; issue stays open; reservation released; counts toward 3-strike stop.
Temporal/Cube lost mid-run	`factory status`	Do not retry-submit (workflow ID is idempotency key); mark manifest `failed` and quarantine.
Admission state corrupt	JSON parse error	Refuse to admit (exit 3), preserve file as `.corrupt-<ts>`, require operator `aep lane reap`.
`policy.data` differs from `HEAD`	`VerifyAgainstHEAD`	Boot FAIL; nothing starts.

11. Test plan

Layer	What is tested	How
Unit	`profile` validation & detection, `policy` scrub/hash, `admission` arithmetic, `gateway` relay verbs, `dispatch.Decide` table (every row of §7)	Table-driven Go tests with fakes for ledger, factory, memoryd, gateway, and memory probe.
Concurrency	Two `Admit` calls racing for the last slot; exactly one wins	Goroutines + a real `lock.FileLock` on a temp dir.
Fault injection	Corrupt admission file; stale fence; launcher killed; broken relay socket	Temp state dir; spawned helper process; assert no double admit, no premature reap.
Parity	`aep boot` vs `boot-gate.sh` output tokens for the same stub environment	Reuse shell gate's stub-PATH technique; diff the `BOOT-GATE:` lines.
Integration (needs runtime)	Lane container respects `--memory`; IPC sockets functional with zero direct network	Runs where a container runtime exists; skipped elsewhere with printed reason.
Cross-platform	New packages compile for all four `dist/bin` targets	CI `GOOS/GOARCH` build matrix; runtime-dependent tests gated by build tag or env.

Per repo practice (see `docs/field-learnings.md`), each new test must be shown to fail against a deliberate break of what it guards before it is accepted; a test that cannot fail is not evidence.

12. Implementation plan (PR-sized tasks)

Each task is a separate branch (`type/scope-short-description`), Conventional-Commit titles, one reviewable deliverable. Tasks T1–T5 need no container runtime and can land on any host.

ID	Task	Files	Acceptance
T0	Feasibility spike: test KVM / nested virt and measure lane RSS on macOS (Colima) and Linux (Podman)	notes in `docs/plan/`	Written result: does `/dev/kvm` exist; peak RSS of ralph lanes. **Gates T9–T10.**
T1	`profile` package + `aep profile detect|show`	`internal/profile/*`, `cmd/aep`	Validate rejects over-committed budgets; nano, small, medium, workstation presets load; KVM detected.
T1b	`gateway` package (`aep gwd` / ghg IPC relay)	`internal/gateway/*`, `cmd/aep`	Socket relay executes `ghg.py`; rate caps and poller coalescing verified without leaking PAT.
T2	`policy` package; HEAD-blob verification	`internal/policy/*`	Matches `boot-gate.sh` behaviour incl. `POLICY-INACTIVE`; tamper test fails gate.
T3	`memoryd` host relay client + `aep boot` full parity	`internal/memoryd/*`, `cmd/aep`	Parity test vs shell gate passes; socket relay forwards to `127.0.0.1:34343`.
T4	`admission` controller	`internal/admission/*`	Race test: one winner for last slot; budget, ceiling, and KVM availability enforced.
T5	`ledger` beads adapter + tier-tag rules	`internal/ledger/*`	Missing-tag policy enforced; `bd` JSON fixtures parsed.
T6	`lane` manifest + fence + Go `claim/release/reap` (runtime driver behind interface, fake in tests)	`internal/lane/*`	Launcher-dead-but-container-alive is refused; socket mounts plumbed.
T7	`dispatch` decision engine + `aep dispatch --dry-run`	`internal/dispatch/*`	Every §7 row covered; quarantine-first on missing KVM; decision records written.
T8	Supervisor `--dispatch` opt-in	`cmd/aep`	Default behaviour byte-identical to today; new path behind the flag.
T9	Compose topology + substrate relays + lane image	`deploy/compose/*`, `Containerfile`	Substrate comes up; lane container enforces memory cap and communicates over sockets. *Needs runtime (T0).*
T10	`factory` client + egress probe verification + `boot --factory`	`internal/factory/*`	Open-egress config rejected; probe digest verified; quarantine path exercised. *Needs Temporal + Cube + KVM.*
T11	Patch re-verification inside disposable microVM; `bd close`	`internal/factory`, `internal/dispatch`	Untrusted patch re-verified in isolated microVM before PR

	+ NeuroStrata inscription		creation.

13. Open items carried from the HLD

- Hardware virtualization availability across environments (T0) — determines whether Tier 2 can run locally or must quarantine/dispatch remotely.
- NeuroStrata socket relay wire protocol vs CLI-only integration (§4.4).
- Doc location: `docs/plan/` (this folder) vs the existing `docs/plans/`.
- Independent adversarial review of this LLD before T9–T11.

AEP LLD v1.1 · updated with PR #133 review remediations and multi-hardware profiles on 2026-11-08 · as-built facts verified against `cmd/aep/main.go` in `develop`; everything marked *new* is unimplemented.